Package 'gofdegrad'

October 22, 2026

Title Goodness-of-Fit Tests for Degradation Models by Similarity and Bootstrap

Version 1.0.0

Description Goodness-of-fit test for degradation models of Zheng, Wang, Chen and Xie (2026, Quality and Reliability Engineering International): the observed trajectories are compared with trajectories simulated under the fitted candidate model through Kolmogorov-Smirnov and Cramer-von Mises type similarity measures, and the critical values are obtained by parametric bootstrap. Seven candidate models are available (Wiener, gamma, inverse Gaussian, variance-gamma and Meixner processes, and Wiener and gamma processes with power-law time transformation), with possibly irregular inspection times common to all trajectories; any other model can be tested by giving its own estimation and simulation functions. The simulations rely on the package 'levysim' and the maximum likelihood estimators on the package 'levyfit'. Level and power studies by Monte Carlo simulation are also provided.

License MIT + file LICENSE

URL https://github.com/rytracetito/Article_zheng_chen_et_al

BugReports https://github.com/rytracetito/Article_zheng_chen_et_al/issues

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Imports ggplot2, levyfit, levysim, Runuran, stats, utils

Suggests SMRD.data, testthat (>= 3.0.0)

Remotes github::rytracetito/levy_packages/levysim, github::rytracetito/levy_packages/levyfit, github::Auburngrads/SMRD.data

Config/testthat/edition 3

Author Amedée Rytrace Tito [aut, cre, cph]

Maintainer Amedée Rytrace Tito <tito.amedee-rytrace@univ-pau.fr>

Contents

gofdegrad-package . . . 2
as.data.frame.gof_test . . . 4
compare_models . . . 5
custom_model . . . 7
fit_model . . . 9
gof_data . . . 11
gof_models . . . 13
gof_test . . . 14
plot.gof_test . . . 17
power_study . . . 18
similarity_stats . . . 20
sim_model . . . 22
sim_path . . . 23

Index **26**

`gofdegrad-package` *gofdegrad: goodness-of-fit tests for degradation models*

Description

`gofdegrad` implements the goodness-of-fit test for degradation models of Zheng, Wang, Chen and Xie (2026). Is a gamma process, a Wiener process or another model adequate for a set of degradation trajectories? The test compares the observed trajectories with trajectories simulated under the fitted model, through similarity measures, and calibrates the decision by parametric bootstrap. It applies to trajectories inspected at the same, possibly irregular, times.

Functions of the package

Test (click on a function name for its help page):

Function	Purpose
`gof_test()`	goodness-of-fit test of one candidate model
`compare_models()`	tests of several candidate models on the same data
`power_study()`	empirical level or power of the test

Models and data:

Function	Purpose
`gof_models()`	the seven available candidate models
`fit_model()`	maximum likelihood estimation of a candidate model
`sim_model()`	simulation of a candidate model
`sim_path()`	simulation of general path models
`custom_model()`	any other candidate model, from its own functions
`gof_data()`	conversion and check of the data
`similarity_stats()`	the four similarity statistics

The result of `gof_test()` is printed as a table, converted to a data frame by as.data.frame() and plotted by plot().

Candidate models

Model	Process	Parameters
`"wiener"`	Wiener with drift	`mu`, `sigma`
`"gamma"`	gamma	shape `a`, rate `b`
`"ig"`	inverse Gaussian	`mu`, `eta`
`"vg"`	variance-gamma	`theta`, `sigma`, `nu`
`"meixner"`	Meixner	`a`, `b`, `d`, `m`
`"wiener_power"`	Wiener, trend μt^r	`mu`, `sigma`, `r`
`"gamma_power"`	gamma, shape function at^r	`a`, `b`, `r`

The first five are Lévy processes, estimated with the package `levyfit` and simulated with the package `levysim`; `gof_models()` gives the law of their increments. Any other model, for instance a general path model, can be tested with `custom_model()`.

Getting started

A test always follows the same three steps.

1. **Prepare the data**: all the trajectories must be inspected at the same times. Simulated data from `levysim` or `sim_model()` can be used directly; real data can be given as a data frame with the columns `id` (trajectory), `t` (time) and `X` (degradation level), or as a matrix with one row per trajectory and the times in `t` (see `gof_data()`). The degradation is measured from time 0.
2. **Test** each candidate model with `gof_test()`, or several at once with `compare_models()`, after `set.seed()` for reproducible results. Use a small number of bootstrap samples `B` to explore, then `B = 1000`.
3. **Read the result**: the p-values and decisions of the four statistics, with plot() for the bootstrap distributions. A non-rejection with few trajectories or few inspections only means that the data are compatible with the model: check the power with `power_study()`.

The test in brief

Let $X_1, \dots, X_m$ be the observed trajectories, inspected at the times $t_1 < \cdots < t_n$. The parameters of the candidate model are estimated by maximum likelihood, and m' trajectories are simulated under the fitted model (the pool, $m' = 100m$ by default). Each observed trajectory is matched with the closest trajectory of the pool, for the distance $\max_j |x_j - y_j|$ (KS type) or $\sum_j (x_j - y_j)^2$ (CvM type), and these smallest distances are summarized by their maximum (`sup`) or their sum (`sum`): four statistics. Their distribution under the null hypothesis is obtained by parametric bootstrap, the parameters being estimated again on each bootstrap sample. See `gof_test()` for the details.

Author(s)

Maintainer: Amedée Rytrace Tito <tito.amedee-rytrace@univ-pau.fr> [copyright holder]

References

Zheng, J., Wang, Y., Chen, Y. and Xie, M. (2026). Automated verification of degradation models using goodness-of-fit test based on similarity measures and parametric bootstrap. *Quality and Reliability Engineering International*. doi:10.1002/qre.70382

Ye, Z.-S. and Xie, M. (2015). Stochastic modelling and analysis of degradation for highly reliable products. *Applied Stochastic Models in Business and Industry*, 31(1), 16-32. doi:10.1002/asmb.2063

Meeker, W. Q., Escobar, L. A. and Pascual, F. G. (2021). *Statistical Methods for Reliability Data*, 2nd edition. Wiley.

See Also

Useful links:

- https://github.com/rytracetito/Article_zheng_chen_et_al
- Report bugs at https://github.com/rytracetito/Article_zheng_chen_et_al/issues
- The packages `levysim` (simulation) and `levyfit` (estimation): https://github.com/rytracetito/levy_packages

Examples

```
# 1. Data: six trajectories of a gamma process, inspected every time unit
set.seed(1)
x <- sim_model(0:30, "gamma", c(a = 2, b = 3), n_traj = 6)
levysim::plot_traj(x)

# 2. Test of the gamma model (small B to keep the example fast)
res <- gof_test(x, "gamma", B = 100, n_pool = 300)
res
plot(res)

# 3. Several candidate models at once
compare_models(x, c("gamma", "ig", "wiener"), B = 50, n_pool = 300, seed = 1)
```

`as.data.frame.gof_test` *Results of a goodness-of-fit test as a data frame*

Description

Converts the result of `gof_test()` into a data frame with one row per statistic: the observed value, the critical value, the p-value and the decision. This is the table printed by `gof_test()`; as a data frame, it can be rounded, saved in a csv file, or gathered with the results of other tests (see also `compare_models()`).

Usage

```
## S3 method for class 'gof_test'
as.data.frame(x, row.names = NULL, optional = FALSE, ...)
```

Arguments

x	Result of gof_test().
row.names, optional, ...	Not used, for compatibility with the generic function.

Value

A data frame with one row per statistic and the columns stat (name of the statistic), statistic (observed value), critical (critical value), p.value and decision ("rejected" or "not rejected").

See Also

gof_test(), compare_models()

Examples

```
# 1. One test as a data frame
set.seed(1)
x <- sim_model(0:20, "wiener", c(mu = 1, sigma = 0.5), n_traj = 5)
res <- gof_test(x, "wiener", B = 50, n_pool = 200)
tab <- as.data.frame(res)
tab

# 2. Rounded values, for a report
tab$statistic <- round(tab$statistic, 3)
tab$critical <- round(tab$critical, 3)
tab

# 3. Several tests gathered in one data frame, with the name of the model
res_wp <- gof_test(x, "wiener_power", B = 50, n_pool = 200)
rbind(cbind(model = "wiener", as.data.frame(res)),
      cbind(model = "wiener_power", as.data.frame(res_wp)))

# 4. Save the table in a csv file
f <- tempfile(fileext = ".csv")
write.csv(tab, f, row.names = FALSE)
```

---

compare_models *Test several candidate models on the same data*

Description

Runs gof_test() for several candidate models on the same data and gathers the estimates, the p-values and the decisions in one table. A model that cannot be fitted to the data (for instance an increasing model when the data contain a decrease) is reported as not testable, with the reason, instead of stopping.

Usage

```
compare_models(
  x,
  models = gof_models()$model,
  t = NULL,
```

```
  B = 1000,
  n_pool = NULL,
  alpha = 0.05,
  seed = NULL
)
```

Arguments

x	Degradation data, in any form accepted by gof_data(): result of a levysim simulation, matrix with a column of times, matrix with one row per trajectory (with t), data frame (id, t, X) or list of data frames. All the trajectories must share the same inspection times, which can be irregular.
models	Candidate models: names of models of gof_models() (by default all of them) or a list mixing names and models built with custom_model().
t	Inspection times, only when x is a matrix with one row per trajectory or a vector (see gof_data()).
B	Number of bootstrap samples, L in the article (default 1000).
n_pool	Number of trajectories of each simulated pool, m' in the article; by default $100m$, the value recommended by the authors, m being the number of observed trajectories.
alpha	Significance level of the test (default 0.05).
seed	If not NULL, the random seed set before each test, so that each result does not depend on the other models tested.

Value

A data frame with one row per model and the columns model, estimate (estimates as text), the p-values KS_sup, KS_sum, CvM_sup, CvM_sum, rejections (number of statistics rejecting H_0, out of 4) and note (reason why a model is not testable, or number of discarded bootstrap samples). The list of the complete test results is in the attribute tests.

See Also

gof_test()

Examples

```
# Inverse Gaussian data tested against three increasing models
set.seed(1)
x <- sim_model(0:30, "ig", c(mu = 0.5, eta = 2), n_traj = 6)
res <- compare_models(x, c("gamma", "ig", "gamma_power"), B = 50, n_pool = 300, seed = 1)
res

# The complete result of one test
attr(res, "tests")$ig

# Data with negative increments: the increasing models are not testable
y <- sim_model(0:30, "wiener", c(mu = 0.5, sigma = 1), n_traj = 6)
compare_models(y, c("wiener", "gamma"), B = 50, n_pool = 300, seed = 1)

# Models still accepted by the four statistics
res$model[which(res$rejections == 0)]
```

```
# A built-in model and a custom model in the same comparison
drift <- custom_model("Wiener with known drift 0.5", "sigma",
  fit = function(X, t) {
    dX <- X - cbind(0, X[, -ncol(X), drop = FALSE])
    dt <- matrix(diff(c(0, t)), nrow(X), length(t), byrow = TRUE)
    c(sigma = sqrt(mean((dX - 0.5 * dt)^2 / dt)))
  },
  simulate = function(theta, t, n) {
   gof_data(sim_model(t, "wiener", c(mu = 0.5, sigma = theta[["sigma"]]), n_traj = n))$X
  })
compare_models(y, list("wiener", drift), B = 50, n_pool = 300, seed = 1)


# Real data: GaAs laser (package SMRD.data), five candidate models
if (requireNamespace("SMRD.data", quietly = TRUE)) {
  laser <- SMRD.data::gaaslaser
  laser <- data.frame(id = laser$unit, t = laser$hours, X = laser$increase)
  compare_models(laser, c("wiener", "gamma", "ig", "wiener_power", "gamma_power"),
                 B = 200, seed = 30)
}
```

`custom_model` *Custom candidate model*

Description

Builds a candidate model from its own estimation and simulation functions, so that any parametric degradation model can be tested by `gof_test()`, fitted by `fit_model()`, simulated by `sim_model()` and studied by `power_study()`: for instance a general path model, a model with random effects, or a Lévy model with another time transformation.

Usage

```
custom_model(name, parameters, fit, simulate, increasing = FALSE)
```

Arguments

`name`	Name of the model, a character string used in the printed results.
`parameters`	Names of the parameters, a character vector.
`fit`	Estimation function `function(X, t)`, where `X` is the $m \times n$ matrix of degradation levels (one row per trajectory, measured from time 0) and `t` the n inspection times after time 0. It returns a named numeric vector of estimates, with the names given in `parameters`.
`simulate`	Simulation function `function(theta, t, n)`, where `theta` is a named numeric vector of parameters, `t` the inspection times after time 0 and `n` the number of trajectories. It returns an $n\times$ `length(t)` matrix of degradation levels, one row per trajectory.
`increasing`	`TRUE` if the trajectories of the model only increase (for information only).

Details

The validity of the test relies on the estimation function: it must be consistent, and it should be the same as the one used in practice for the model, because the bootstrap reproduces it on every bootstrap sample. The two functions are called about $2B$ times by `gof_test()`, so they should be fast.

Value

An object of class `gof_model`, to be given as argument `model` of `gof_test()`, `fit_model()`, `sim_model()` and `power_study()`.

See Also

`gof_models()` for the built-in models, `sim_path()` for general path models.

Examples

```
# A linear general path model: X_i(t) = phi_i t + error, with
# phi_i ~ N(mu, s_phi^2) and measurement errors N(0, s_eps^2)
linear <- custom_model(
  name = "linear path",
  parameters = c("mu", "s_phi", "s_eps"),
  fit = function(X, t) {
    slopes <- apply(X, 1, function(x) sum(x * t) / sum(t^2))
    res <- X - outer(slopes, t)
    c(mu = mean(slopes), s_phi = stats::sd(slopes),
      s_eps = sqrt(sum(res^2) / (length(res) - nrow(X))))
  },
  simulate = function(theta, t, n) {
    phi <- stats::rnorm(n, theta[["mu"]], theta[["s_phi"]])
    outer(phi, t) + matrix(stats::rnorm(n * length(t), 0, theta[["s_eps"]]), n)
  }
)
linear

# Data simulated from this model, fitted, then tested
set.seed(1)
x <- sim_model(1:20, linear, c(mu = 0.5, s_phi = 0.1, s_eps = 0.2), n_traj = 6)
fit_model(x, linear)
gof_test(x, linear, B = 50, n_pool = 200)

# The same data tested against a Wiener process
gof_test(x, "wiener", B = 50, n_pool = 200)

# A Wiener process whose trend and variance follow different time scales,
# X(t) = mu t^0.5 + sigma B(t^0.25), which is not among the built-in
# models: given the two time functions, the estimators are explicit
Lambda <- function(t) t^0.5
tau <- function(t) t^0.25
wiener_tt <- custom_model(
  name = "Wiener, trend t^0.5, variance t^0.25",
  parameters = c("mu", "sigma"),
  fit = function(X, t) {
    dX <- X - cbind(0, X[, -ncol(X), drop = FALSE])
    dL <- matrix(diff(Lambda(c(0, t))), nrow(X), length(t), byrow = TRUE)
    dT <- matrix(diff(tau(c(0, t))), nrow(X), length(t), byrow = TRUE)
```

```
    mu <- sum(dX * dL / dT) / sum(dL^2 / dT)
    c(mu = mu, sigma = sqrt(mean((dX - mu * dL)^2 / dT)))
  },
  simulate = function(theta, t, n) {
    x <- levysim::sim_wiener(c(0, t), mu = theta[["mu"]], sigma = theta[["sigma"]],
                             Lambda = Lambda, tau = tau, n_traj = n)
    gof_data(x)$X
  }
)
t <- levysim::inspection_times("increasing", n = 29, tmax = 35, digits = 2)$time
y <- sim_model(t, wiener_tt, c(mu = 2, sigma = 0.78), n_traj = 6)
fit_model(y, wiener_tt)
gof_test(y, wiener_tt, B = 50, n_pool = 300)
```

`fit_model` *Maximum likelihood estimation of a candidate model*

Description

Estimates the parameters of one of the candidate models of `gof_models()` (or of a model built with `custom_model()`) from one or several degradation trajectories inspected at the same, possibly irregular, times. This is the estimation step of `gof_test()`, applied once to the observed data and once to each bootstrap sample.

Usage

```
fit_model(x, model, t = NULL)
```

Arguments

`x`	Degradation data, in any form accepted by `gof_data()`: result of a `levysim` simulation, matrix with a column of times, matrix with one row per trajectory (with `t`), data frame (`id`, `t`, `X`) or list of data frames.
`model`	Name of the model, one of `"wiener"`, `"gamma"`, `"ig"`, `"vg"`, `"meixner"`, `"wiener_power"`, `"gamma_power"` (see `gof_models()`), or a model built with `custom_model()`.
`t`	Inspection times, only when `x` is a matrix with one row per trajectory or a vector (see `gof_data()`).

Details

Estimators. For the five Lévy models the estimators of the package `levyfit` are used, applied to the increments of all the trajectories pooled together:

- Wiener: closed form, `levyfit::wiener_mle()`; the returned `sigma` is the square root of the maximum likelihood estimate of σ^2;
- gamma: root of one equation, `levyfit::gamma_mle()`;
- inverse Gaussian: closed form, `levyfit::ig_mle()`;
- variance-gamma: EM algorithm then quasi-Newton, `levyfit::vg_mle()`;
- Meixner: profile likelihood, `levyfit::meixner_mle()`.

For the two power-law models, the likelihood is profiled in r:

- `"wiener_power"`: for a given r, with $\Delta\Lambda_j = t_j^r - t_{j-1}^r$, the estimators are explicit,

$$\hat{\mu}(r) = \frac{\sum_j (\Delta\Lambda_j/\Delta t_j) S_j}{m \sum_j \Delta\Lambda_j^2/\Delta t_j}, \qquad \hat{\sigma}^2(r) = \frac{1}{mn} \sum_{i,j} \frac{(\Delta x_{ij} - \hat{\mu}(r)\Delta\Lambda_j)^2}{\Delta t_j},$$

  where S_j is the sum of the increments of the j-th interval; the profile likelihood is maximal where $\hat{\sigma}^2(r)$ is minimal;
- `"gamma_power"`: for a given r, the process is a homogeneous gamma process in the time t^r, estimated by `levyfit::gamma_mle()`.

r is searched by `stats::optimize()` in $[0.01, 20]$, on the time scale t/t_n for numerical stability; a warning is given when the estimate is at a boundary of this interval (for instance with a single trajectory, whose trend is poorly identified).

Increasing models. The gamma, inverse Gaussian and power-law gamma models only increase: a negative increment in the data gives an error. An increment exactly equal to 0 is usually a rounding effect: a very small increment, added to a large level, disappears in the computer. It is replaced by the machine precision times the largest level, an upper bound of its true value, with a warning.

Estimation failures. When the maximum likelihood estimator does not exist for the data (for instance a variance-gamma model fitted to increasing data, whose likelihood is maximal at the gamma limit), `levyfit` gives a warning and the estimates are `NA`.

Value

A named numeric vector with the estimates of the parameters of the model (see `gof_models()`), with the attribute `model` (name of the model).

References

Zheng, J., Wang, Y., Chen, Y. and Xie, M. (2026). Automated verification of degradation models using goodness-of-fit test based on similarity measures and parametric bootstrap. *Quality and Reliability Engineering International*. doi:10.1002/qre.70382

See Also

`gof_models()`, `sim_model()`, `gof_test()`, and the package `levyfit` for confidence intervals and more estimation options.

Examples

```
# 1. Six trajectories of a gamma process, inspected every time unit
set.seed(1)
x <- sim_model(0:50, "gamma", c(a = 2, b = 3), n_traj = 6)
fit_model(x, "gamma")

# 2. Several models fitted to the same data
fit_model(x, "ig")
fit_model(x, "wiener")

# 3. Irregular inspection times (common to all the trajectories)
t <- levysim::inspection_times("increasing", n = 30, tmax = 50)$time
x <- sim_model(t, "wiener_power", c(mu = 0.5, sigma = 0.3, r = 0.6), n_traj = 10)
fit_model(x, "wiener_power")

# 4. Gamma process with power-law shape function
```

```
x <- sim_model(0:40, "gamma_power", c(a = 1, b = 2, r = 1.5), n_traj = 10)
fit_model(x, "gamma_power")

# 5. A matrix with one row per trajectory and the times given in t
X <- matrix(c(0.6, 1.3, 2.2, 3.1,
              0.4, 1.1, 1.9, 2.4,
              0.7, 1.2, 2.5, 3.3), nrow = 3, byrow = TRUE)
fit_model(X, "gamma", t = c(1, 2, 3.5, 5))

# 6. A decrease makes the increasing models impossible
try(fit_model(c(0, 1, 0.8, 2), "gamma", t = c(0, 1, 2, 3)))

# 7. Real data: GaAs laser (package SMRD.data)
if (requireNamespace("SMRD.data", quietly = TRUE)) {
  laser <- SMRD.data::gaaslaser
  laser <- data.frame(id = laser$unit, t = laser$hours, X = laser$increase)
  fit_model(laser, "wiener")
  fit_model(laser, "wiener_power")
}
```

`gof_data` *Degradation data in the form used by the package*

Description

Converts degradation data given in any of the forms accepted by the package into a single internal form: the inspection times after time 0 and the matrix of degradation levels, one row per trajectory. This function is called by `gof_test()`, `fit_model()` and `similarity_stats()`; it is exported so that the data can be checked before a test.

Usage

```
gof_data(x, t = NULL)
```

Arguments

`x`	Degradation data, in one of the following forms: • the result of a simulation function of the package `levysim` (for instance `levysim::sim_gamma()`), of `sim_model()` or of `sim_path()`; • a numeric matrix whose first column contains the inspection times and the other columns the degradation levels of the trajectories (the format of `levysim` and `levyfit`), when `t` is not given; • a numeric matrix with one row per trajectory and one column per inspection time, when `t` is given; • a numeric vector of degradation levels of one trajectory, with `t`; • a data frame with the columns `id` (trajectory), `t` (inspection time) and `X` (degradation level), in any row order, for instance read from a csv file or returned by `levysim::traj_df()`; the trajectories are sorted by `id`; • a list of data frames, one per trajectory, whose first column contains the inspection times and second column the degradation levels.
`t`	Inspection times, only when `x` is a matrix with one row per trajectory or a vector: one time per column of `x` (or per element).

Details

Common inspection times. The test of Zheng et al. (2026) compares trajectories inspected at the same times, so all the trajectories must share the same inspection times. These times can be irregular.

Origin of the degradation. The models start from $X(0) = 0$. When the data contain time 0, the level at time 0 of each trajectory is subtracted from its levels, so that the degradation is measured from the initial state (for instance from the initial length of a crack). When the first inspection is after time 0, the degradation is assumed to be 0 at time 0: the first increment goes from 0 at time 0 to the first observed level, and it weighs on the estimation of the models.

Value

An object of class `gof_data`: a list with

- `t`: the n inspection times after time 0, strictly increasing;
- `X`: the $m \times n$ matrix of degradation levels, measured from the level at time 0, one row per trajectory.

See Also

`gof_test()`, `fit_model()`

Examples

```
# 1. Simulated data from levysim: a matrix with the column t
set.seed(1)
x <- levysim::sim_gamma(seq(0, 10, by = 1), a = 2, b = 3, n_traj = 4)
d <- gof_data(x)
d
d$t
dim(d$X)

# 2. One row per trajectory, with the times given separately
X <- matrix(c(0.5, 1.1, 1.8,
              0.4, 0.9, 1.5), nrow = 2, byrow = TRUE)
gof_data(X, t = c(1, 2, 4))

# 3. A data frame (id, t, X), as read from a csv file: the levels at
#    time 0 (here 0.9) are subtracted
df <- data.frame(id = rep(1:2, each = 3), t = rep(c(0, 1, 2), 2),
                 X = c(0.9, 1.0, 1.2, 0.9, 1.1, 1.4))
gof_data(df)$X

# 4. A list of data frames (t, X), one per trajectory
liste <- list(data.frame(t = c(0, 5, 10), X = c(0, 1.2, 2.9)),
              data.frame(t = c(0, 5, 10), X = c(0, 0.8, 2.1)))
gof_data(liste)

# 5. Real data in long format: GaAs laser data (package SMRD.data)
if (requireNamespace("SMRD.data", quietly = TRUE)) {
  laser <- SMRD.data::gaaslaser
  d <- gof_data(data.frame(id = laser$unit, t = laser$hours, X = laser$increase))
  dim(d$X)
}
```

`gof_models` *Candidate models available in the package*

Description

Lists the seven candidate models that can be tested by `gof_test()`, fitted by `fit_model()` and simulated by `sim_model()`, with their parameters, the law of their increments and the methods used for the estimation and the simulation.

Usage

```
gof_models()
```

Details

All the models start from $X(0) = 0$ and have independent increments. The first five are Lévy processes (homogeneous in time); the last two have a power-law time transformation, t^r, which gives a concave ($r < 1$) or convex ($r > 1$) mean degradation path.

- `"wiener"`: Wiener process with drift, $X(t) = \mu t + \sigma B(t)$; increments can be negative.
- `"gamma"`: gamma process with shape a and **rate** b (scale $1/b$), $E[X(t)] = at/b$; increasing.
- `"ig"`: inverse Gaussian process with the parametrization of Wang and Xu (2010), $E[X(t)] = \mu t$, $Var(X(t)) = \mu^3 t/\eta$; increasing.
- `"vg"`: variance-gamma process, a Brownian motion with drift θ and diffusion σ evaluated at a gamma time of variance ν per unit of time; increments can be negative.
- `"meixner"`: Meixner process with parameters $a > 0$, $-\pi < b < \pi$, $d > 0$ and real m; increments can be negative.
- `"wiener_power"`: Wiener process with power-law trend, $X(t) = \mu t^r + \sigma B(t)$, the non-homogeneous Wiener model of Zheng et al. (2026): the trend is transformed, not the variance.
- `"gamma_power"`: gamma process with shape function at^r and rate b; increasing.

The increasing models (gamma, inverse Gaussian, power-law gamma) cannot be fitted to data with a negative increment. An increment exactly equal to 0, due to the rounding of the levels, is replaced by a tiny positive value with a warning (see `fit_model()`).

Value

A data frame with one row per model and the columns `model` (name to give to the other functions), `process`, `parameters`, `increasing` (`TRUE` for the models with increasing trajectories), `law` (law of the increment between times $s < t$), `estimation` and `simulation`.

References

Zheng, J., Wang, Y., Chen, Y. and Xie, M. (2026). Automated verification of degradation models using goodness-of-fit test based on similarity measures and parametric bootstrap. *Quality and Reliability Engineering International*. doi:10.1002/qre.70382

Wang, X. and Xu, D. (2010). An inverse Gaussian process model for degradation data. *Technometrics*, 52(2), 188-197. doi:10.1198/TECH.2009.08197

See Also

`gof_test()`, `fit_model()`, `sim_model()`, `custom_model()` to test another model.

Examples

```
# All the models
gof_models()

# Names and parameters only
gof_models()[, c("model", "parameters")]

# Models with increasing trajectories
gof_models()$model[gof_models()$increasing]
```

---

gof_test *Goodness-of-fit test by similarity measures and parametric bootstrap*

Description

Tests whether degradation trajectories follow a candidate model, with the method of Zheng, Wang, Chen and Xie (2026) (their Algorithm 1). The observed trajectories are compared with a large pool of trajectories simulated under the fitted model through four similarity statistics, and the distribution of these statistics under the null hypothesis is obtained by parametric bootstrap.

Usage

```
gof_test(
  x,
  model,
  t = NULL,
  B = 1000,
  n_pool = NULL,
  alpha = 0.05,
  progress = FALSE
)
```

Arguments

`x`	Degradation data, in any form accepted by `gof_data()`: result of a `levysim` simulation, matrix with a column of times, matrix with one row per trajectory (with `t`), data frame (`id`, `t`, `X`) or list of data frames. All the trajectories must share the same inspection times, which can be irregular.
`model`	Candidate model under the null hypothesis: the name of a model of `gof_models()` (`"wiener"`, `"gamma"`, `"ig"`, `"vg"`, `"meixner"`, `"wiener_power"`, `"gamma_power"`) or a model built with `custom_model()`.
`t`	Inspection times, only when `x` is a matrix with one row per trajectory or a vector (see `gof_data()`).
`B`	Number of bootstrap samples, L in the article (default 1000).
`n_pool`	Number of trajectories of each simulated pool, m' in the article; by default $100m$, the value recommended by the authors, m being the number of observed trajectories.

alpha	Significance level of the test (default 0.05).
progress	TRUE to display a progress bar during the bootstrap.

Details

Null hypothesis. H_0: the m trajectories are independent realizations of the candidate model $M(\theta)$, for some unknown parameter θ.

Algorithm (Zheng et al., 2026, Algorithm 1).

1. Estimate $\hat{\theta}$ by maximum likelihood from the data (see `fit_model()`).
2. Simulate a pool of m' trajectories under $M(\hat{\theta})$ at the inspection times of the data, and compute the four statistics S between the data and the pool (see `similarity_stats()`).
3. For $b = 1, \ldots, B$: simulate a bootstrap sample of m trajectories under $M(\hat{\theta})$, estimate $\hat{\theta}_b^*$ from it, simulate a pool of m' trajectories under $M(\hat{\theta}_b^*)$ and compute the statistics S_b^*. Re-estimating the parameters on each bootstrap sample reproduces the effect of the estimation on the statistics.
4. For each statistic, the critical value is the empirical quantile of order $1 - \alpha$ of the S_b^*, the p-value is the proportion of $S_b^* \geq S$, and H_0 is rejected when S exceeds the critical value.

Choice of the statistics. The `sup` statistics react to one trajectory far from the model, the `sum` statistics to a misfit shared by all the trajectories. The authors recommend the `sup` statistics for an automated use; looking at the four is more informative.

Bootstrap failures. When the estimation fails on a bootstrap sample (for instance no maximum likelihood estimate for a variance-gamma or Meixner sample), this sample is discarded and counted in `n_failed`; the critical values and p-values use the other samples. Warnings raised on the bootstrap samples are not shown. A failure on the observed data stops the test with an error explaining why the model cannot be fitted.

Computing time. The test calls the estimation and the simulation of the model about $2B$ times. With the default values it takes a few seconds for the Wiener, gamma and inverse Gaussian models, and one or a few minutes for the variance-gamma, Meixner and power-law models. Use a small `B` (50 to 200) to explore, then `B = 1000` for the final results, after `set.seed()` for reproducibility.

Power. With few trajectories or few inspections, the test has little power: a model that does not fit the data well, for instance a homogeneous model fitted to concave paths, may not be rejected because its estimated variance is inflated. A non-rejection means that the data are compatible with the model, not that the model is validated. Use `power_study()` to assess the power of the test in a given design.

Value

An object of class `gof_test`, a list with

- `model`, `process`: name of the candidate model and of its process;
- `estimate`: estimates of the parameters from the data;
- `statistic`: observed values of the four statistics;
- `critical`: critical values;
- `p.value`: p-values;
- `reject`: TRUE when H_0 is rejected, for each statistic;
- `alpha`, `B`, `n_pool`: settings of the test;
- `n_failed`: number of discarded bootstrap samples;

- `m`, `t`: number of trajectories and inspection times of the data;
- `boot`: matrix of the bootstrap statistics, one row per sample.

Printing the object shows a summary table; `as.data.frame()` gives this table as a data frame and `plot()` draws the bootstrap distributions.

References

Zheng, J., Wang, Y., Chen, Y. and Xie, M. (2026). Automated verification of degradation models using goodness-of-fit test based on similarity measures and parametric bootstrap. *Quality and Reliability Engineering International*. doi:10.1002/qre.70382

See Also

`compare_models()` to test several models on the same data, `power_study()` for the level and the power of the test, `gof_models()`, `fit_model()`, `similarity_stats()`, `plot.gof_test()`.

Examples

```
# Small values of B and n_pool to keep the examples fast;
# use the default values (B = 1000, n_pool = 100 m) in practice.

# 1. Gamma data tested against the gamma model: H0 is true
set.seed(1)
x <- sim_model(0:30, "gamma", c(a = 2, b = 3), n_traj = 6)
res <- gof_test(x, "gamma", B = 100, n_pool = 300)
res

# 2. The elements of the result
res$estimate
res$p.value
res$reject
as.data.frame(res)

# 3. Bootstrap distributions with the observed values (solid lines)
#    and the critical values (dashed lines)
plot(res)

# 4. Concave Wiener paths tested against the homogeneous Wiener model
x <- sim_model(1:30, "wiener_power", c(mu = 2, sigma = 0.3, r = 0.5), n_traj = 6)
gof_test(x, "wiener", B = 100, n_pool = 300)
gof_test(x, "wiener_power", B = 50, n_pool = 300)

# 5. Irregular inspection times common to all the trajectories
t <- levysim::inspection_times("increasing", n = 20, tmax = 40)$time
x <- sim_model(t, "ig", c(mu = 0.5, eta = 2), n_traj = 8)
gof_test(x, "ig", B = 100, n_pool = 400)

# 6. Data given as a matrix, one row per trajectory, with the times
X <- gof_data(x)$X
gof_test(X, "ig", t = gof_data(x)$t, B = 50, n_pool = 400)

# 7. Gamma process with power-law shape function
x <- sim_model(1:25, "gamma_power", c(a = 1, b = 2, r = 1.5), n_traj = 6)
gof_test(x, "gamma_power", B = 30, n_pool = 300)
```

```
# 8. A progress bar for long tests
res <- gof_test(x, "gamma", B = 30, n_pool = 300, progress = TRUE)


# 9. Variance-gamma and Meixner models (slower estimation)
x <- sim_model(0:40, "vg", c(theta = 0.5, sigma = 1, nu = 0.5), n_traj = 6)
gof_test(x, "vg", B = 100)
gof_test(x, "wiener", B = 100)
y <- sim_model(0:100, "meixner", c(a = 1, b = 0.3, d = 1.5, m = 0.2), n_traj = 6)
gof_test(y, "meixner", B = 50)

# 10. Real data: GaAs laser (package SMRD.data); the homogeneous Wiener
#    model is rejected, as in Zheng et al. (2026)
if (requireNamespace("SMRD.data", quietly = TRUE)) {
  laser <- SMRD.data::gaaslaser
  laser <- data.frame(id = laser$unit, t = laser$hours, X = laser$increase)
  set.seed(30)
  gof_test(laser, "wiener", B = 200)
}
```

`plot.gof_test` *Plot the bootstrap distributions of a goodness-of-fit test*

Description

Draws, for each of the four statistics, the histogram of the bootstrap statistics with the observed value (solid line) and the critical value (dashed line). H_0 is rejected by a statistic when the solid line is to the right of the dashed line.

Usage

```
## S3 method for class 'gof_test'
plot(x, ...)
```

Arguments

x	Result of `gof_test()`.
...	Not used.

Value

A `ggplot` object, which can be modified with the usual `ggplot2` functions.

See Also

`gof_test()`

Examples

```
# 1. H0 true: the observed values (solid lines) stay within the
#    bootstrap distributions, left of the critical values (dashed lines)
set.seed(1)
x <- sim_model(0:20, "gamma", c(a = 2, b = 3), n_traj = 5)
res <- gof_test(x, "gamma", B = 100, n_pool = 250)
plot(res)

# 2. H0 false: concave paths tested against the homogeneous Wiener model;
#    the observed values are far to the right of the critical values
y <- sim_model(1:20, "wiener_power", c(mu = 2, sigma = 0.2, r = 0.4), n_traj = 5)
plot(gof_test(y, "wiener", B = 100, n_pool = 250))

# 3. Change the title and the axis labels
plot(res) + ggplot2::labs(title = "Gamma model, bootstrap distributions",
                          x = "Value of the statistic")

# 4. Save the plot
f <- tempfile(fileext = ".pdf")
ggplot2::ggsave(f, plot(res), width = 8, height = 5)
```

---

power_study *Empirical level or power of the goodness-of-fit test*

Description

Repeats gof_test() on n_rep independent data sets generated by a function, and returns the rejection rate of each of the four statistics. When the data are generated under the candidate model, this rate is the empirical level of the test, to be compared with alpha; otherwise it is the empirical power against the generating model.

Usage

```
power_study(
  generate,
  model,
  n_rep = 100,
  B = 200,
  n_pool = NULL,
  alpha = 0.05,
  failure = c("reject", "exclude"),
  progress = FALSE
)
```

Arguments

generate	Function without argument that returns one data set, in any form accepted by gof_data(), for instance function() sim_model(0:50, "gamma", c(a = 2, b = 3), n_traj = 6).
model	Candidate model under the null hypothesis, as in gof_test().
n_rep	Number of data sets (default 100).

B	Number of bootstrap samples of each test (default 200).
n_pool	Size of the simulated pools; by default $100m$.
alpha	Significance level of the test (default 0.05).
failure	What to do with a data set to which the candidate model cannot be fitted, for instance increasing models with negative increments: "reject" (default) counts it as a rejection by the four statistics, since such data are not consistent with the model; "exclude" removes it from the rates.
progress	TRUE to display a progress bar.

Details

The empirical level and power are Monte Carlo estimates: their standard error, $\sqrt{p(1-p)/n}$ for a rate p computed on n data sets, is printed with the results. It is about 0.022 for a level of 0.05 with 100 data sets, and 0.007 with 1000 data sets.

The computing time is about n_rep times that of one test. A study with n_rep = 1000 and B = 1000, as in the article, takes hours; start with small values. The number of data sets to which the model could not be fitted is always reported, so that a power due to these failures can be told apart from a power due to the test.

Value

An object of class gof_power, a list with

- rate: rejection rate of each statistic;
- se: Monte Carlo standard error of each rate;
- rejections: logical matrix of the decisions, one row per data set (rows of the excluded data sets removed);
- n_rep, n_failed: number of data sets and number of data sets to which the model could not be fitted;
- model, alpha, B, failure: settings of the study.

References

Zheng, J., Wang, Y., Chen, Y. and Xie, M. (2026). Automated verification of degradation models using goodness-of-fit test based on similarity measures and parametric bootstrap. *Quality and Reliability Engineering International*. doi:10.1002/qre.70382

See Also

gof_test(), sim_model(), sim_path()

Examples

```
# Very small values to keep the examples fast: in practice use
# n_rep = 1000 and B = 1000, as in the article.
t <- 0:20

# 1. Empirical level: data generated under the gamma model
set.seed(1)
power_study(function() sim_model(t, "gamma", c(a = 2, b = 3), n_traj = 6),
            model = "gamma", n_rep = 10, B = 30, n_pool = 150)
```

```
# 2. Power of the Wiener model against concave Wiener paths
power_study(function() sim_model(t, "wiener_power", c(mu = 1, sigma = 0.3, r = 0.5), n_traj = 6),
            model = "wiener", n_rep = 10, B = 30, n_pool = 150)

# 3. Power of the Wiener model against linear general paths
power_study(function() sim_path(t, "linear", phi_mean = 0.2, phi_sd = 0.05,
                                sigma_eps = 0.05, n_traj = 6),
            model = "wiener", n_rep = 10, B = 30, n_pool = 150)

# 4. Wiener data tested against the gamma model: negative increments make
#    the gamma model impossible; the failures are counted as rejections
#    by default, or excluded
gen <- function() sim_model(t, "wiener", c(mu = 0.5, sigma = 0.5), n_traj = 6)
power_study(gen, model = "gamma", n_rep = 10, B = 30, n_pool = 150)

# 5. Wiener data with a strong drift: some data sets have only positive
#    increments and can be fitted by the gamma model; with
#    failure = "exclude", the rate is the power of the test on them only
gen <- function() sim_model(t, "wiener", c(mu = 2, sigma = 0.5), n_traj = 2)
p <- power_study(gen, model = "gamma", n_rep = 10, B = 30, n_pool = 150,
                 failure = "exclude")
p

# 6. The elements of the result
p$rate
p$se
p$n_failed
head(p$rejections)

# 7. Level with irregular inspection times and a progress bar
t_irr <- levysim::inspection_times("dec-inc", n = 15, tmax = 20)$time
power_study(function() sim_model(t_irr, "ig", c(mu = 0.5, eta = 2), n_traj = 4),
            model = "ig", n_rep = 10, B = 30, n_pool = 150, progress = TRUE)
```

`similarity_stats` *Similarity statistics between trajectories and a simulated pool*

Description

Computes the four test statistics of Zheng et al. (2026) between a set of m trajectories and a pool of m' trajectories simulated under the candidate model, all inspected at the same times. Each trajectory is matched with its most similar trajectory of the pool; the statistics summarize these smallest distances by their maximum (`sup`) or their sum (`sum`).

Usage

```
similarity_stats(x, pool)
```

Arguments

`x`	Trajectories to test: a numeric matrix with one row per trajectory and one column per inspection time, or any data form accepted by `gof_data()` (for instance a `levysim` simulation).
`pool`	Pool of simulated trajectories, in the same form as `x` and at the same inspection times.

Details

For two trajectories $X = (x_1, \ldots, x_n)$ and $Y = (y_1, \ldots, y_n)$ observed at the same times, the two similarity measures are

$$D_{KS}(X, Y) = \max_j |x_j - y_j|, \qquad D_{CvM}(X, Y) = \sum_j (x_j - y_j)^2,$$

by analogy with the Kolmogorov-Smirnov and Cramér-von Mises statistics. For each trajectory X_i of `x`, let $d_i = \min_k D(X_i, Y_k)$ be its distance to the most similar trajectory of the pool. The statistics are

$$S_{sup} = \max_i d_i, \qquad S_{sum} = \sum_i d_i,$$

for each of the two measures. Large values mean that some (`sup`) or all (`sum`) observed trajectories have no similar trajectory in the pool, that is, are unlikely under the candidate model. With a single trajectory, the `sup` and `sum` statistics are equal.

Value

A named numeric vector of length 4: `KS_sup`, `KS_sum`, `CvM_sup`, `CvM_sum`.

References

Zheng, J., Wang, Y., Chen, Y. and Xie, M. (2026). Automated verification of degradation models using goodness-of-fit test based on similarity measures and parametric bootstrap. *Quality and Reliability Engineering International*. doi:10.1002/qre.70382

See Also

`gof_test()`, which computes these statistics on the data and on the bootstrap samples.

Examples

```
# 1. Six gamma trajectories against a pool of 600 gamma trajectories
set.seed(1)
t <- 1:30
x <- sim_model(t, "gamma", c(a = 2, b = 3), n_traj = 6)
pool <- sim_model(t, "gamma", c(a = 2, b = 3), n_traj = 600)
similarity_stats(x, pool)

# 2. The same trajectories against a pool from another model: larger values
pool_ig <- sim_model(t, "ig", c(mu = 2 / 3, eta = 0.5), n_traj = 600)
similarity_stats(x, pool_ig)

# 3. Small matrices, one row per trajectory
X <- matrix(c(1, 2, 3,
              1, 3, 4), nrow = 2, byrow = TRUE)
Y <- matrix(c(1, 2, 2,
              2, 3, 5,
              0, 1, 3), nrow = 3, byrow = TRUE)
similarity_stats(X, Y)
```

`sim_model` *Simulation of a candidate model*

Description

Simulates exactly one or several degradation trajectories of one of the candidate models of `gof_models()` (or of a model built with `custom_model()`) at given inspection times, common to all the trajectories. This is the simulation step of `gof_test()`, used to generate the bootstrap samples and the pools of simulated trajectories.

Usage

```
sim_model(t, model, theta, n_traj = 1)
```

Arguments

`t`	Inspection times, strictly increasing, with or without time 0 (the trajectories start from 0 at time 0).
`model`	Name of the model (see `gof_models()`) or a model built with `custom_model()`.
`theta`	Parameters of the model: a named numeric vector or list, with the names given in `gof_models()` (for instance `c(a = 2, b = 3)` for the gamma model). The result of `fit_model()` can be given directly.
`n_traj`	Number of trajectories (default 1).

Details

The five Lévy models and the two power-law models are simulated by the package `levysim` (exact simulation of the independent increments), except the Meixner model: `levysim::sim_meixner()` draws the increments one at a time, which is too slow for the thousands of trajectories of a bootstrap. The Meixner increments of a same time step are therefore drawn together, with the same law (numerical inversion of the distribution function with `Runuran`, parameters a, b, $d\Delta t$, $m\Delta t$).

The parametrizations are those of `gof_models()`: in particular the gamma model uses the **rate** b, as in `levysim` and `levyfit`.

Value

An object of class `levysim`: a matrix whose first column `t` contains the inspection times (time 0 included) and the columns `X1`, ..., `Xn` the simulated levels, equal to 0 at time 0. It can be plotted with `levysim::plot_traj()`, converted to a data frame with `levysim::traj_df()`, and given directly to `fit_model()` and `gof_test()`.

See Also

`gof_models()`, `fit_model()`, `sim_path()` for general path models, and the package `levysim` for more simulation options (different inspection times per trajectory, other time transformations).

Examples

```
# 1. Ten trajectories of a Wiener process, inspected every 0.5 time unit
set.seed(1)
x <- sim_model(seq(0, 10, by = 0.5), "wiener", c(mu = 0.5, sigma = 0.8), n_traj = 10)
x
levysim::plot_traj(x)

# 2. The seven models, with the same inspection times
t <- 0:20
sim_model(t, "gamma", c(a = 2, b = 3), n_traj = 3)
sim_model(t, "ig", c(mu = 0.5, eta = 2), n_traj = 3)
sim_model(t, "vg", c(theta = 0.5, sigma = 1, nu = 0.5), n_traj = 3)
sim_model(t, "meixner", c(a = 1, b = 0.3, d = 1.5, m = 0.2), n_traj = 3)
sim_model(t, "wiener_power", c(mu = 1, sigma = 0.3, r = 0.5), n_traj = 3)
sim_model(t, "gamma_power", c(a = 1, b = 2, r = 1.5), n_traj = 3)

# 3. Irregular inspection times, more and more frequent
t <- levysim::inspection_times("increasing", n = 25, tmax = 50)$time
x <- sim_model(t, "gamma", c(a = 2, b = 3), n_traj = 8)
levysim::plot_traj(x, "Gamma process, irregular inspections")

# 4. Simulate from estimated parameters (parametric bootstrap)
theta <- fit_model(x, "gamma")
y <- sim_model(t, "gamma", theta, n_traj = 8)

# 5. All the data in one data frame, for instance to save them
head(levysim::traj_df(y))
```

`sim_path` *Simulation of general path degradation models*

Description

Simulates trajectories of a general path model (Algorithm 3 of Zheng et al., 2026): a deterministic degradation path with random effects, specific to each unit, plus independent measurement errors. These models are the usual alternatives to the stochastic process models in a power study (see `power_study()`); they can be tested themselves with `custom_model()`.

Usage

```
sim_path(
  t,
  shape = c("linear", "power", "exponential"),
  phi_mean,
  phi_sd,
  sigma_eps,
  omega = NULL,
  n_traj = 1
)
```

Arguments

`t`	Inspection times, strictly increasing, with or without time 0.
`shape`	Shape of the path: `"linear"`, `"power"` or `"exponential"` (see Details).
`phi_mean`, `phi_sd`	Mean and standard deviation of the normal random effects: one number each for the linear and exponential paths, a vector of length 2 each for the power path (ϕ_1 then ϕ_2).
`sigma_eps`	Standard deviation of the measurement errors, $\sigma_\varepsilon \geq 0$.
`omega`	Rate of the exponential path, a positive number; used only by `shape = "exponential"`.
`n_traj`	Number of trajectories (default 1).

Details

Each unit k has its own random effect ϕ_k, drawn from a normal distribution, and is observed with independent measurement errors $\varepsilon_{kj} \sim N(0, \sigma_\varepsilon^2)$:

$$X_k(t_j) = g(t_j; \phi_k) + \varepsilon_{kj},$$

with

- `"linear"`: $g(t;\phi) = \phi t$;
- `"power"`: $g(t;\phi) = \phi_1 t^{\phi_2}$, with independent ϕ_1 and ϕ_2;
- `"exponential"`: $g(t;\phi) = \phi(e^{\omega t} - 1)$.

The level at time 0 is 0, without measurement error. The normality of the random effects is an assumption of the package: the article does not specify their distribution.

Value

An object of class `levysim`: a matrix whose first column `t` contains the inspection times (time 0 included) and the columns `X1`, ..., `Xn` the simulated levels. It can be plotted with `levysim::plot_traj()` and given to `fit_model()` and `gof_test()`.

References

Zheng, J., Wang, Y., Chen, Y. and Xie, M. (2026). Automated verification of degradation models using goodness-of-fit test based on similarity measures and parametric bootstrap. *Quality and Reliability Engineering International*. doi:10.1002/qre.70382

Lu, C. J. and Meeker, W. Q. (1993). Using degradation measures to estimate a time-to-failure distribution. *Technometrics*, 35(2), 161-174. doi:10.1080/00401706.1993.10485038

See Also

`sim_model()` for the stochastic process models, `power_study()`, `custom_model()`.

Examples

```
t <- 0:50
set.seed(1)

# 1. Linear paths with random slopes
x <- sim_path(t, "linear", phi_mean = 0.2, phi_sd = 0.05, sigma_eps = 0.1, n_traj = 6)
```

```
levysim::plot_traj(x)

# 2. Power paths: concave when phi_2 < 1
x <- sim_path(t, "power", phi_mean = c(1, 0.5), phi_sd = c(0.2, 0.05),
              sigma_eps = 0.1, n_traj = 6)
levysim::plot_traj(x, "Power paths")

# 3. Exponential paths
x <- sim_path(t, "exponential", phi_mean = 0.2, phi_sd = 0.05, sigma_eps = 0.1,
              omega = 0.05, n_traj = 6)
levysim::plot_traj(x, "Exponential paths")

# 4. Irregular inspection times
t <- levysim::inspection_times("dec-inc", n = 20, tmax = 50)$time
x <- sim_path(t, "linear", phi_mean = 0.2, phi_sd = 0.05, sigma_eps = 0.1, n_traj = 6)
head(levysim::traj_df(x))

# 5. Is a linear path model with random slopes taken for a Wiener process?
gof_test(x, "wiener", B = 50, n_pool = 200)
```

Index

as.data.frame(), *3*, *16*
as.data.frame.gof_test, 4

compare_models, 5
compare_models(), *2–5*, *16*
custom_model, 7
custom_model(), *2*, *3*, *6*, *9*, *14*, *22–24*

fit_model, 9
fit_model(), *2*, *7*, *8*, *11–16*, *22*, *24*

gof_data, 11
gof_data(), *2*, *3*, *6*, *9*, *14*, *18*, *20*
gof_models, 13
gof_models(), *2*, *3*, *6*, *8–10*, *14*, *16*, *22*
gof_test, 14
gof_test(), *2–14*, *17–19*, *21*, *22*, *24*
gofdegrad *(*gofdegrad-package*)*, 2
gofdegrad-package, 2

levyfit::gamma_mle(), *9*, *10*
levyfit::ig_mle(), *9*
levyfit::meixner_mle(), *9*
levyfit::vg_mle(), *9*
levyfit::wiener_mle(), *9*
levysim::plot_traj(), *22*, *24*
levysim::sim_meixner(), *22*
levysim::traj_df(), *22*

plot(), *3*, *16*
plot.gof_test, 17
plot.gof_test(), *16*
power_study, 18
power_study(), *2*, *3*, *7*, *8*, *15*, *16*, *23*, *24*

sim_model, 22
sim_model(), *2*, *3*, *7*, *8*, *10*, *11*, *13*, *14*, *19*, *24*
sim_path, 23
sim_path(), *2*, *8*, *11*, *19*, *22*
similarity_stats, 20
similarity_stats(), *2*, *11*, *15*, *16*
stats::optimize(), *10*